Lexora

Intelligent Legal Case Document Analysis and Legal Research Platform

Session 2025 – 2027

Supervisor: Dr. Aftab Akram, Assistant Professor

Program: BSIT (Post ADP)

Shift: Morning

Submitted By

1. Osama Eijaz (mtf25004943)
2. Shoaib Ajmal (mtf25004913)
3. Abdul Barr (mtf25004947)
4. Abdullah Naseer (mtf25004952)

DEPARTMENT OF INFORMATION SCIENCES

UNIVERSITY OF EDUCATION, LAHORE

Campus: Township

October 07, 2026

Declaration

We declare that our Final Year Project (FYP) proposal, “Lexora: Intelligent Legal Case Document Analysis and Legal Research Platform”, is our own work, based on our own research and development. We have not copied it from anywhere, and its similarity index is below 20%.

Submission Date: _____

Signature: _____
Name: OSAMA EIJAZ
Date: _____

Signature: _____
Name: SHOAIB AJMAL
Date: _____

Signature: _____
Name: ABDUL BARR
Date: _____

Signature: _____
Name: ABDULLAH NASEER
Date: _____

Table of Contents

1. Introduction 1

2. Related Work 1

   2.1 Historical Background 1

   2.2 Related Prior Work and Projects 1

   2.3 Current Achievements in Development 2

3. Problem Description 2

   3.1 Core Features and Benefits 2

   3.2 Design Considerations and Reliability 2

4. Aims and Objectives 3

   4.1 Aim 3

   4.2 Objectives 3

5. Domain of the Project 3

6. Project Scope 4

   6.1 Scope and Key Deliverables 4

   6.2 Out of Scope (Aspects Not Considered) 4

   6.3 System Assumptions 4

7. Methodology 5

   7.1 System Architecture and Technical Pipeline 5

   7.2 Implementation Steps 6

8. Tools and Technology 6

   8.1 Hardware Requirements 7

   8.2 Core AI and RAG Engine 7

   8.3 Application Stack and Frameworks 8

9. Project Milestones (Gantt Chart) 9

10. References 9

1. Introduction

Lexora is a web application we are building to make life easier for lawyers and their clients. Legal work means reading a lot of long and sensitive papers, such as contracts, lease agreements, non-compete clauses, and wills. Going through them by hand takes a lot of time. Many cloud-based AI tools can speed this up, but they need you to upload your case files to someone else's servers. For law firms, that is a real problem, because it puts client confidentiality and attorney-client privilege at risk.

So we took a different route. Lexora runs a Retrieval-Augmented Generation (RAG) system entirely on the firm's own computer, using Ollama (Llama 3) and ChromaDB. Reading documents, indexing them, and answering questions all happen locally, so no file ever leaves the machine.

To make the answers reliable, Lexora uses a Hybrid Search Engine that mixes dense vector embeddings with BM25 keyword search. It also has an Interactive PDF Citation Viewer that highlights the exact part of the document an answer came from, so a lawyer can check it quickly. On top of this, Lexora can flag risky clauses in a contract, build a timeline for a case, draft documents from templates, and show a dashboard that measures how accurate the answers are (context precision and faithfulness). With strict data isolation and full audit logging, Lexora gives legal teams a practical, scalable, and privacy-friendly way to work.

2. Related Work

2.1 Historical Background

Early legal software was fairly simple. It used relational databases to keep records and keyword-matching methods like BM25 to find documents (Robertson & Zaragoza, 2009). These worked for finding exact words, but they could not understand meaning. Things changed with transformer models (Vaswani et al., 2017) and language models built on them, such as BERT (Devlin et al., 2019), which gave computers a much better grasp of language. Even so, using a large language model (LLM) on its own in legal work is risky, because it can make things up and may expose private data. Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), helps with the first problem by making the model base its answers on real documents.

2.2 Related Prior Work and Projects

- **RAG for Legal Case Retrieval:** Wahidur et al. (2025) proposed specialized RAG systems such as LQ-RAG, which combine hybrid vector search with domain fine-tuning. Their results show that when answers are tied to verified legal text, the model hallucinates less and finds the right material more often than plain keyword search.
- **Privacy-Preserving Local RAG Frameworks:** Velamala (2026) built LocalRAG, an offline framework that uses local vector indexing (FAISS) together with BM25 search.

Because it runs fully on local hardware, it shows that smaller quantized open-weights models can still give precise answers while keeping data private and avoiding cloud APIs.

- **On-Device LLM Systems for Legal Workflows:** Open-source models such as Llama 2 (Touvron et al., 2023) and Llama 3 (Meta AI, 2024) can now run on local tools like Ollama (Ollama Project, 2024). This shows that legal drafting and analysis can be done locally without any risk of data leaking to the cloud.

2.3 Current Achievements in Development

Today's legal AI is moving toward local, hybrid designs. Open-weights models, vector stores like ChromaDB, and local evaluation tools like Ragas (Es et al., 2023) make it possible to build legal assistants that are both trustworthy and confidential. Lexora builds on this progress. It places an offline, air-gapped RAG pipeline inside a complete practice management system, so case handling, contract analysis, and drafting all happen in one place without going online.

3. Problem Description

Lexora is an on-premise case management and document intelligence web application. It is designed to solve a common conflict in legal work: lawyers want to work faster, but they cannot risk their clients' confidentiality. Contracts, wills, and court filings are complex, and reading them manually is slow. Commercial cloud AI tools are fast, but they send confidential files to outside servers, which goes against attorney-client privilege. Lexora keeps everything local by using Ollama (Llama 3) and ChromaDB, so client data stays completely private.

3.1 Core Features and Benefits

- **Local RAG & Hybrid Search:** Mixes vector embeddings with BM25 keyword search to answer legal questions accurately, with exact source citations.
- **Contract Risk & Drafting:** Checks contracts for missing or risky clauses and creates customizable legal notices or petitions that can be exported to Word or PDF.
- **Integrated Case Management:** Takes care of case registration, hearing schedules, and separate access for lawyers and clients.

3.2 Design Considerations and Reliability

- **Data Isolation:** No third-party APIs are used, so nothing can leak to the cloud.
- **Hallucination Prevention:** Answers are based on retrieved text, and citations are highlighted in the document so lawyers can verify them right away.
- **Local Hardware Optimization:** Since the AI runs on the firm's own machine, speed depends on that hardware, so response time has to be managed carefully.

Reliability matters a lot here. A wrong legal summary or an exposed client file can have serious legal consequences. By basing every answer on the uploaded documents and running

everything in an isolated local setup, Lexora aims to be dependable and respectful of privacy rules.

4. Aims and Objectives

4.1 Aim

Our aim is to build a privacy-first legal management platform that runs on the firm's own system. It should speed up document analysis and case handling without putting client confidentiality at risk and without the ongoing cost of cloud APIs.

4.2 Objectives

- Build a fully local RAG pipeline with Ollama (Llama 3) and ChromaDB for safe document search and contract risk detection.
- Combine hybrid retrieval (dense vectors + BM25) with highlighted citations, so hallucinations are kept to a minimum and every answer can be checked.
- Simplify daily legal work by bringing together case scheduling, separate lawyer and client access, and automatic Word/PDF draft export.
- Make Lexora available as a responsive web app that works smoothly on laptops, PCs, tablets, and Android devices.
- Measure the quality of retrieval and answers using a local RAG Evaluation Metrics Dashboard (context precision, context recall, and faithfulness).

5. Domain of the Project

Lexora is a full-stack web development project that also uses applied Artificial Intelligence (a local RAG setup) and focuses on data security and privacy. In short, it covers three main areas:

- **Full-Stack Web Application Development:** Creating a responsive web portal (React frontend, FastAPI/Django backend) that works on laptops, PCs, tablets, and Android devices, with Role-Based Access Control (RBAC) and separate dashboards for clients and lawyers.
- **Artificial Intelligence & Natural Language Processing (NLP):** Building an on-premise Retrieval-Augmented Generation (RAG) system with local LLMs (Ollama / Llama 3), a ChromaDB vector store, and hybrid search.
- **Data Security & Information Privacy:** Setting up a zero-cloud, fully isolated environment so all data stays local and client confidentiality is protected.

6. Project Scope

6.1 Scope and Key Deliverables

This project will deliver a complete, working on-premise web application called Lexora. It brings together everyday legal case management and a local Retrieval-Augmented Generation (RAG) engine. The main deliverables are:

- Separate dashboards for lawyers and clients, unique Case ID generation, hearing schedules, and automatic case timelines.
- Local reading of PDF/DOCX files, vector indexing with ChromaDB, and hybrid (dense + BM25) search.
- Question answering with a local LLM (Ollama/Llama 3), an interactive PDF citation viewer, contract risk analysis, legal debate simulation, and template-based drafting.
- Editable Word/PDF exports, audit logging, and a RAG evaluation metrics dashboard.
- A responsive web app that runs in modern browsers on laptops, PCs, tablets, and Android devices, with the same features on every screen size.

6.2 Out of Scope (Aspects Not Considered)

To finish the project realistically within 12 weeks, we will not work on the following:

- **Third-Party / Public Cloud AI Services:** We will not connect to outside cloud services (such as OpenAI, Anthropic, or cloud vector stores), so that data stays fully isolated.
- **Automated Court E-Filing & Government API Integration:** Lexora will not connect to official court databases or government portals to file cases automatically.
- **Complex Multi-Model Training / Fine-Tuning:** We will not train models from scratch or do heavy fine-tuning. We rely on local RAG indexing and open-source models (Ollama / Llama 3).
- **Real-Time Active Video Conferencing:** There will be no live video calls between lawyers and clients.
- **Mobile App Development:** Lexora is a single responsive web app that works in the browser on laptops, PCs, tablets, and Android devices.

6.3 System Assumptions

The project depends on the following assumptions:

- **Hardware Capability:** The computer running the server is an ordinary modern machine with at least 8 GB RAM (16 GB recommended). A GPU is optional and only makes answers faster, because we use a small 4-bit quantized model that runs well on the CPU. Other devices only need a browser.

- **Document Integrity:** Uploaded legal files (PDF/DOCX) are readable digital documents, not badly damaged files or handwritten pages.
- **Network Access:** Tablets and Android devices connect to Lexora through the firm's local network (Wi-Fi/LAN) using a modern browser such as Chrome.
- **Single-Node On-Premise Deployment:** Lexora will be installed on the law firm's own computer or private server, not on a multi-region cloud setup.

7. Methodology

We will build Lexora step by step, one component at a time. The basic system comes first, and the advanced AI features are added on top once the base is stable. Everything runs on-premise, so all data stays local.

7.1 System Architecture and Technical Pipeline

The system works in four stages, shown in Figure 7.1.

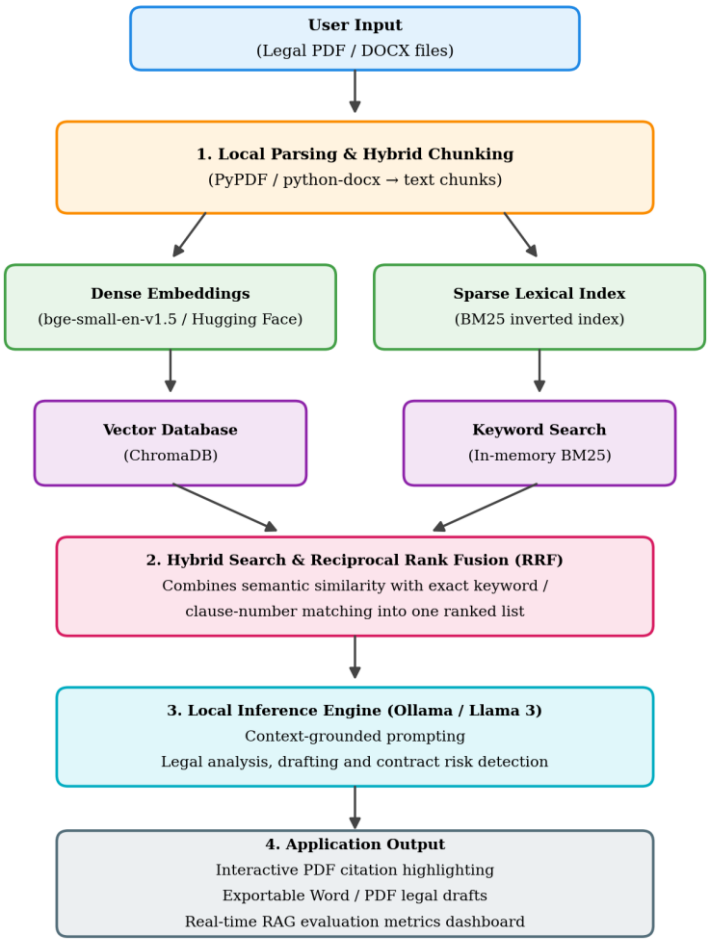


Figure 7.1: Lexora system architecture and technical pipeline

7.2 Implementation Steps

Phase 1: Environment and Core Case Management Setup

- Set up the web portal with React (frontend) and FastAPI/Django (backend), using SQLite/PostgreSQL to store case data.
- Add Role-Based Access Control (RBAC) to separate the Lawyer and Client portals, generate unique Case IDs (for example, CASE-2026-001), and schedule hearings and deadlines.
- Make the layout responsive and test it in browsers on laptop, PC, tablet, and Android devices.

Phase 2: Local Processing and Hybrid RAG Retrieval Engine

- Create local upload pipelines for legal contracts, wills, and petitions (PDF/DOCX).
- Split documents into chunks locally, store their dense embeddings in ChromaDB, and build a BM25 index alongside it to catch exact keywords and clause numbers.
- Use Reciprocal Rank Fusion (RRF) to merge the best results from both search methods.

Phase 3: Specialized Local Legal AI Tools

- Send the merged context to a locally running Ollama (Llama 3) model.
- Write custom prompts for contract risk detection, case timeline generation, legal debate simulation, and template-based petition and notice drafting.
- Add export features using server-side libraries (python-docx / ReportLab).

Phase 4: Citation Rendering, Evaluation and System Hardening

- Connect an interactive PDF viewer (react-pdf) that takes citation markers from the AI's answers and highlights the matching text in the source.
- Build an internal RAG Evaluation Metrics Dashboard (using local Ragas) that keeps track of Context Precision, Context Recall, and Faithfulness.
- Add audit logging and tighten security to confirm that no data leaves the local environment.

8. Tools and Technology

Below are the hardware, software, and technologies we need to build and run Lexora. We chose light tools on purpose, so Lexora can run on a normal office computer. The hardware below is needed only for the Lexora server. Users simply open Lexora in a browser on their laptop, PC, tablet, or Android device.

8.1 Hardware Requirements

Table 8.1: Minimum hardware specification

Component	Specification	Purpose
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (6-core minimum)	Handles web requests, document reading, indexing, and runs the small language model.
System Memory (RAM)	8 GB minimum (16 GB recommended)	Holds the 4-bit model (about 2–3 GB), vector caches, BM25 indexes, and the application.
Graphics Card (GPU)	Optional: any NVIDIA GPU with 4 GB VRAM or more	Makes answers faster when available. Without it, the CPU does the work.
Storage	256 GB SSD	Gives fast read/write speed for quick searches and for loading model files.

8.2 Core AI and RAG Engine

Table 8.2: Core AI and RAG technologies

Technology	Role in Lexora
Ollama (Local LLM Server)	Tool used to run and manage open-source language models fully on our own machine.
Llama 3.2 (3B Instruct, 4-bit quantized)	A small, light language model (about 2 GB). It answers legal questions, runs debate simulations, and writes drafts on an ordinary computer, without any cloud API.
ChromaDB (Vector Database)	Open-source database that stores document embeddings locally and finds similar text quickly.
Sentence Transformers (bge-small-en-v1.5)	A lightweight Hugging Face model that runs locally and turns text chunks into 384-dimensional vectors.
rank-bm25 (Sparse Lexical Search)	Python library for BM25 keyword search. It helps find exact clause numbers and legal terms.
LangChain / LlamaIndex	Frameworks that connect the RAG pipeline, manage prompts, and merge search results using Reciprocal Rank Fusion (RRF).
Ragas	Evaluation library that measures context precision, context recall, and faithfulness for the dashboard.

8.3 Application Stack and Frameworks

8.3.1 Frontend

Table 8.3: Frontend technologies

Technology	Role in Lexora
React.js	Single-page web framework used to build the interactive lawyer and client portals.
react-pdf / PDF.js	Libraries that show PDFs inside the browser and highlight the cited text.
Tailwind CSS / Bootstrap 5	Styling frameworks that make dashboards and pages look good on any screen size.

8.3.2 Backend

Table 8.4: Backend technologies

Technology	Role in Lexora
Python (FastAPI / Django)	API framework that handles business logic, case workflows, file uploads, and communication with the LLM.
SQLite / PostgreSQL	Relational database for case records, hearing schedules, Case ID indexing, and user access data.
python-docx / ReportLab	Server-side libraries that create editable Word and PDF versions of legal drafts.

9. Project Milestones (Gantt Chart)

We plan to finish the project in 12 weeks, split into six overlapping phases. These run from requirements and design all the way to integration, testing, and defense preparation. Figure 9.1 shows the full schedule.

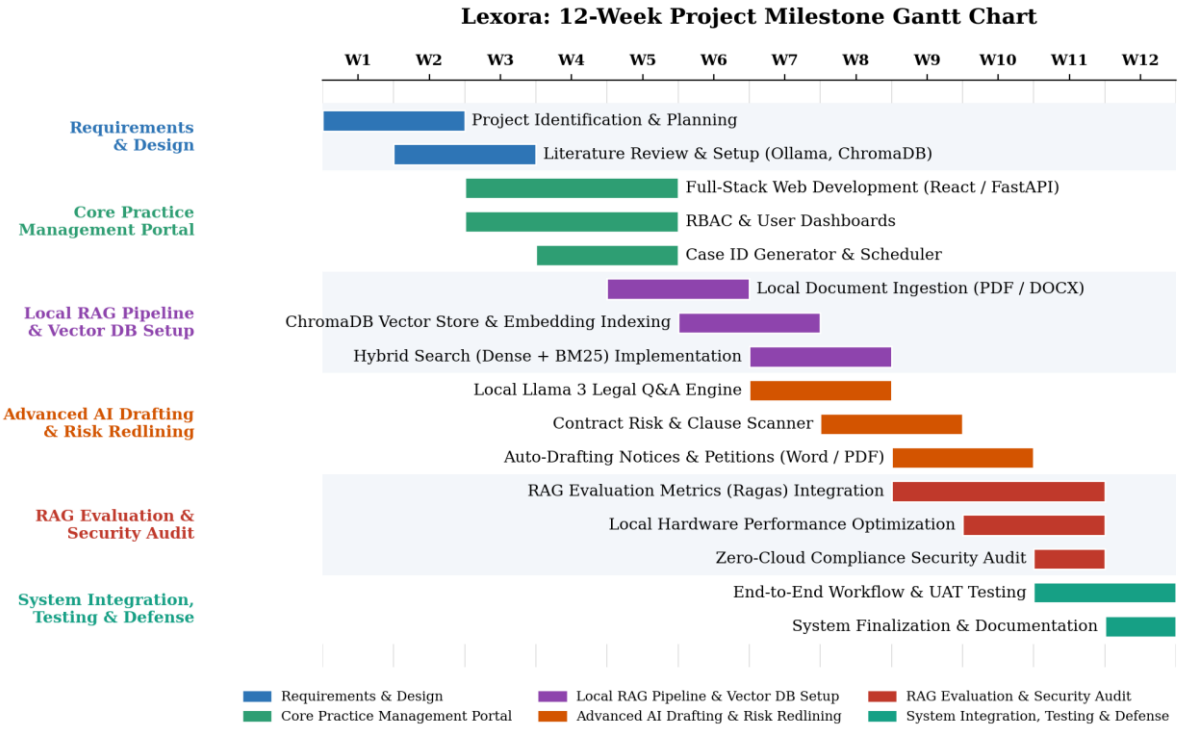


Figure 9.1: Lexora 12-week project milestone Gantt chart

10. References

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Vol. 1, pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation. *arXiv*. https://doi.org/10.48550/arXiv.2309.15217

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 9459–9474). Curran Associates, Inc.

- Meta AI. (2024). *Llama 3 community license agreement and technical report*. Meta Platforms, Inc. https://ai.meta.com/llama/

- Ollama Project. (2024). *Ollama: Get up and running with Llama 3, Mistral, and other large language models locally* [Computer software]. https://ollama.com
- Robertson, S. E., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333–389. https://doi.org/10.1561/1500000019
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., … Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv*. https://doi.org/10.48550/arXiv.2307.09288
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 5998–6008). Curran Associates, Inc.